PENTEST EM APLICAÇÃO WEB (DVWA)

Exploração de Falhas de Autenticação com Hydra

Relatório Técnico de Segurança

Autora: Kelwri Jheine

Data: 30 de março de 2026

SUMÁRIO

1. Introdução 03

2. Metodologia 03

3. Escopo 04

4. Ferramentas Utilizadas 04

5. Ambiente de Teste 04

6. Descobertas 05

6.1 Serviço SSH Exposto (Porta 22) 05

6.1.1 *Evidências* 05

6.2 Serviço HTTP sem Criptografia (Porta 80) 07

6.2.1 *Evidências* 07

6.3 Falha de Autenticação por Força Bruta 08

6.3.1 *Evidências* 09

6.4 Análise de Requisições HTTP (DevTools) 10

6.4.1 *Evidências* 10

6.5 Análise de Cabeçalhos HTTP e Cookies 11

6.5.1 *Evidências* 12

6.6 Criação de Wordlist 13

6.6.1 *Evidências* 14

6.7 Ataque de Força Bruta com Hydra 15

6.7.1 *Evidências* 15

7. Conclusão 17

1. INTRODUÇÃO

Este relatório apresenta os resultados de um teste de intrusão realizado em ambiente controlado, com o objetivo de identificar vulnerabilidades na aplicação avaliada.

Os testes demonstraram a obtenção de credenciais válidas na ausência de mecanismos adequados de proteção, evidenciando alto risco de comprometimento do sistema.

Classificação de risco geral
ALTO

2. METODOLOGIA

A análise foi conduzida seguindo etapas padrão de testes de intrusão:

- Reconhecimento
- Enumeração
- Análise
- Exploração
- Pós-exploração

3. ESCOPO

- **Ambiente:** Laboratório controlado
- **Alvo:** Serviço web exposto na porta 80
- **Aplicação:** DVWA (Damn Vulnerable Web Application)
- **Tipo de teste:** Teste de intrusão em aplicação web (foco em autenticação)

4. FERRAMENTAS UTILIZADAS

- Nmap
- Hydra
- Kiwi Browser (DevTools – análise de requisições e cookies)

5. AMBIENTE DE TESTE

Aplicação alvo:	DVWA
Conectividade:	VPN (TryHackMe)
Virtualização:	Máquina virtual (TryHackMe)
Dispositivo de ataque:	Smartphone Android
Ambiente de execução:	UserLAnd (Linux em espaço de usuário)

Os testes foram realizados em ambiente isolado e controlado, garantindo segurança e reprodutibilidade dos resultados.

6. DESCOBERTAS

6.1 Serviço SSH Exposto (Porta 22)

Descrição

A porta 22/tcp encontra-se aberta, executando o serviço SSH (OpenSSH 6.6.1p1). A versão identificada está desatualizada e pode conter vulnerabilidades conhecidas. A exposição do serviço permite conexões remotas e amplia a superfície de ataque, especialmente na ausência de mecanismos adicionais de proteção.

Impacto

A disponibilidade do SSH pode ser explorada por meio de tentativas de autenticação indevida ou exploração de falhas conhecidas. Em caso de comprometimento, há risco de acesso remoto, execução de comandos e movimentação lateral.

Criticidade

Médio

6.1.1 Evidências:

Comando utilizado: nmap -sV -sC 10.XX.XXX.XXX

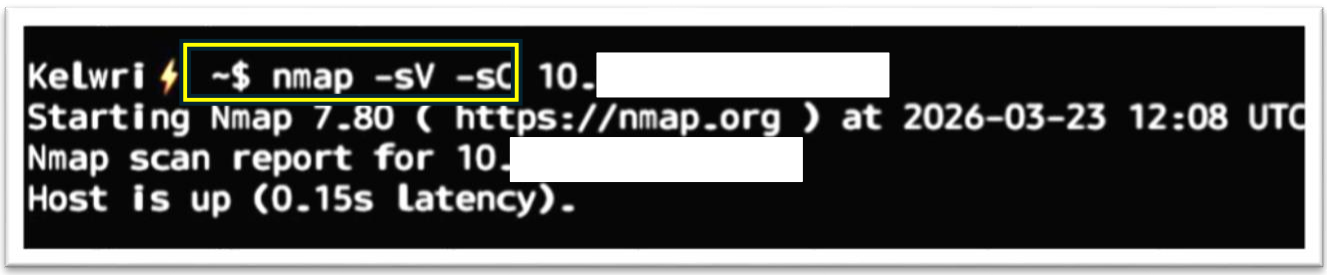

Figura 1 – Execução do comando Nmap para enumeração de serviços no alvo.

Informações coletadas a partir da evidência:

- Ferramenta utilizada: Nmap 7.80

- Tipo de varredura: Detecção de versão (-sV) e scripts padrão (-sC)

- Host ativo: resposta com latência de 0.15s

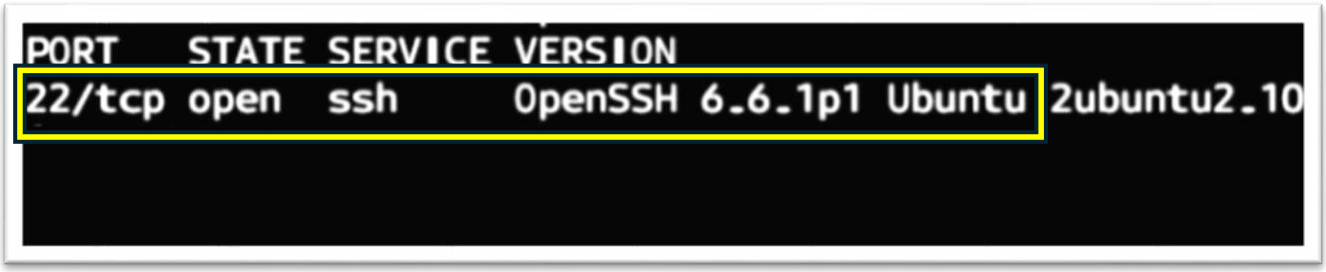

Figura 2 – Varredura com Nmap identificando a porta 22/tcp aberta e o serviço SSH (OpenSSH 6.6.1p1).

Informações coletadas a partir da evidência:

- Porta aberta: 22/tcp

- Serviço identificado: SSH

- Versão do serviço: OpenSSH 6.6.1p1

- Sistema operacional associado: Ubuntu

Medidas

a. Restringir acesso por IP, permitindo conexões apenas de origens autorizadas.

b. Utilizar autenticação por chave pública em substituição a senhas.

c. Manter o serviço atualizado para correção de vulnerabilidades conhecidas.

6.2 Serviço HTTP sem Criptografia (Porta 80)

Descrição

A porta 80/tcp encontra-se aberta, expondo o serviço HTTP sem criptografia. Isso permite interceptação de dados e facilita a análise da aplicação por agentes maliciosos.

Impacto

A ausência de proteção adequada possibilita ataques automatizados, manipulação de requisições e exploração de falhas, comprometendo a confidencialidade e integridade das informações.

Criticidade

Alto

6.2.1 Evidências:

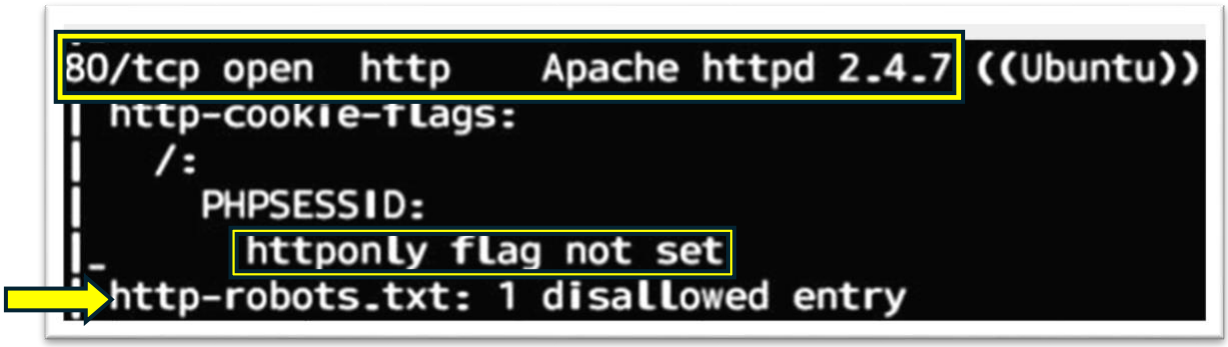

Figura 3 – Resultado da enumeração do serviço HTTP (porta 80/tcp), evidenciando o servidor Apache 2.4.7 (Ubuntu), ausência da flag HttpOnly no cookie PHPSESSID e presença do arquivo robots.txt com entradas restritas.

Informações coletadas a partir da evidência:

- Porta aberta: 80/tcp
- Estado do serviço: ativo (open)

• Serviço identificado: HTTP

• Servidor web: Apache httpd 2.4.7 (Ubuntu)

• Cookie de sessão: PHPSESSID

• Configuração insegura: ausência da flag HttpOnly

• Arquivo robots.txt detectado

• Presença de diretivas restritivas (disallowed), indicando possíveis diretórios sensíveis

Medidas

a. Implementar mecanismos anti-automação, como CAPTCHA e limitação de requisições.

b. Validar e sanitizar todas as entradas no servidor.

c. Adotar HTTPS para proteger os dados em trânsito.

d. Reduzir a exposição de informações em cabeçalhos HTTP.

e. Configurar cookies com flag HttpOnly.

6.3 Falha de Autenticação por Força Bruta

Descrição

Foi identificada a ausência de controles contra tentativas repetidas de autenticação, permitindo ataques de força bruta por meio de múltiplas combinações de credenciais.

Impacto

A vulnerabilidade possibilita o comprometimento de contas válidas, acesso não autorizado e execução de ações em nome de usuários legítimos.

Criticidade

Alto

6.3.1 Evidências:

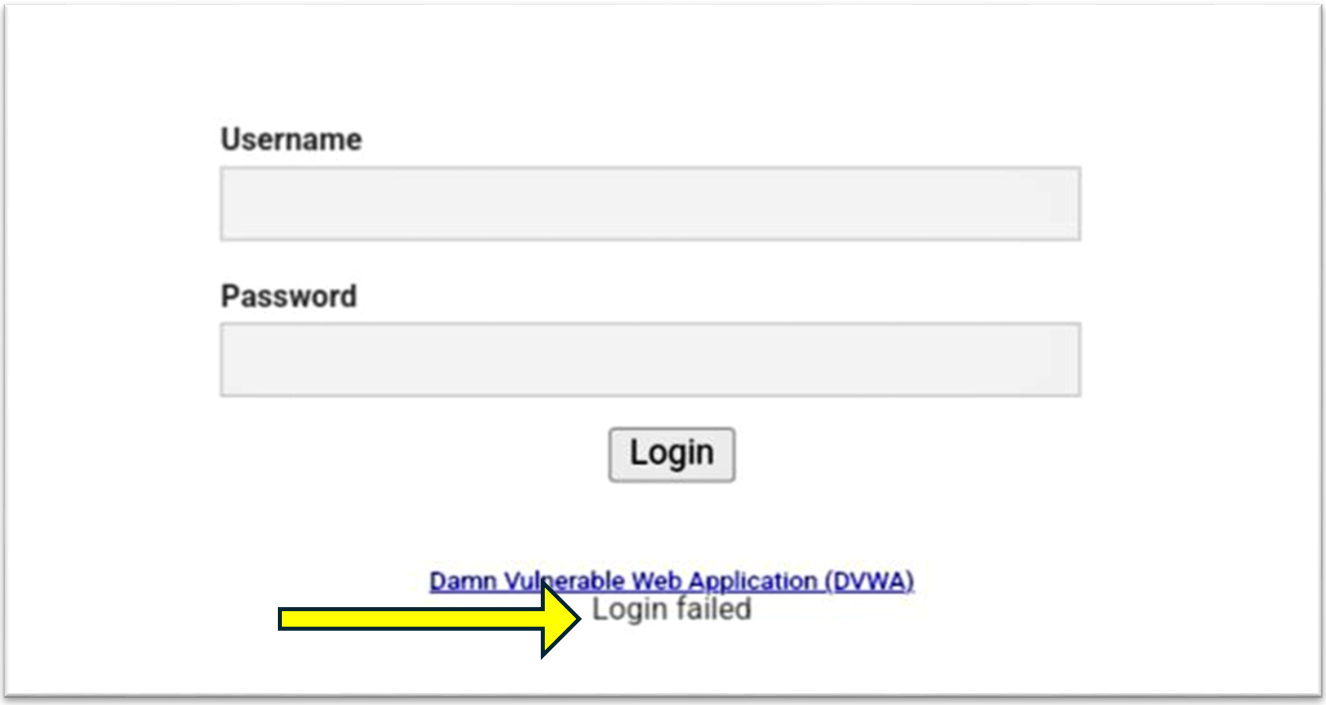

Figura 4 – Página de login da DVWA com mensagem “Login failed”, confirmando validação de credenciais.

Informações coletadas a partir da evidência:

- Página de login contendo campos Username e Password

- Botão Login

- Mensagem exibida: “Login failed”

Medidas

a. Limitar tentativas de login por intervalo de tempo.

b. Implementar autenticação multifator (MFA).

c. Monitorar e bloquear comportamentos suspeitos.

6.4 Análise de Requisições HTTP (DevTools)

Descrição

A análise das requisições HTTP via DevTools permitiu observar o comportamento da aplicação, incluindo respostas, recursos carregados e padrões de comunicação.

Impacto

Essas informações podem ser utilizadas para mapear endpoints, manipular requisições e explorar falhas lógicas.

Criticidade

Médio

6.4.1 Evidências:

Nome	Status	Tipo	Iniciador
login.php	200	document	Outro
login.css	200	stylesheet	login.php
login_logo.png	200	png	login.php
favicon.ico	200	vnd.microsoft.icon	Outro

Figura 5 – Monitoramento das requisições HTTP via DevTools (aba Network), evidenciando respostas 200 e carregamento de recursos da aplicação.

Recursos identificados:

- /login.php – 200 (document)

Página principal de autenticação da aplicação. Responsável pelo processamento das credenciais inseridas pelo usuário e ponto central do fluxo de login.

- /login.css – 200 (stylesheet)

Arquivo de estilo responsável pela formatação visual da página de login, definindo layout, cores e organização dos elementos.

- /login_logo.png – 200 (image)

Recurso gráfico utilizado na interface da página de autenticação, contribuindo para a identidade visual da aplicação.

- /favicon.ico – 200 (icon)

Ícone exibido na aba do navegador, utilizado para identificação visual da aplicação.

Medidas

a. Validar todas as requisições no lado do servidor.

b. Limitar o volume de requisições por IP ou usuário.

c. Implementar proteção contra CSRF com uso de tokens.

6.5 Análise de Cabeçalhos HTTP e Cookies

Descrição

Foram identificadas informações expostas em cabeçalhos HTTP e cookies sem atributos de segurança adequados.

Impacto

Esses dados podem facilitar ataques direcionados e permitir o sequestro de sessão.

Criticidade

Alto

6.5.1 Evidências:

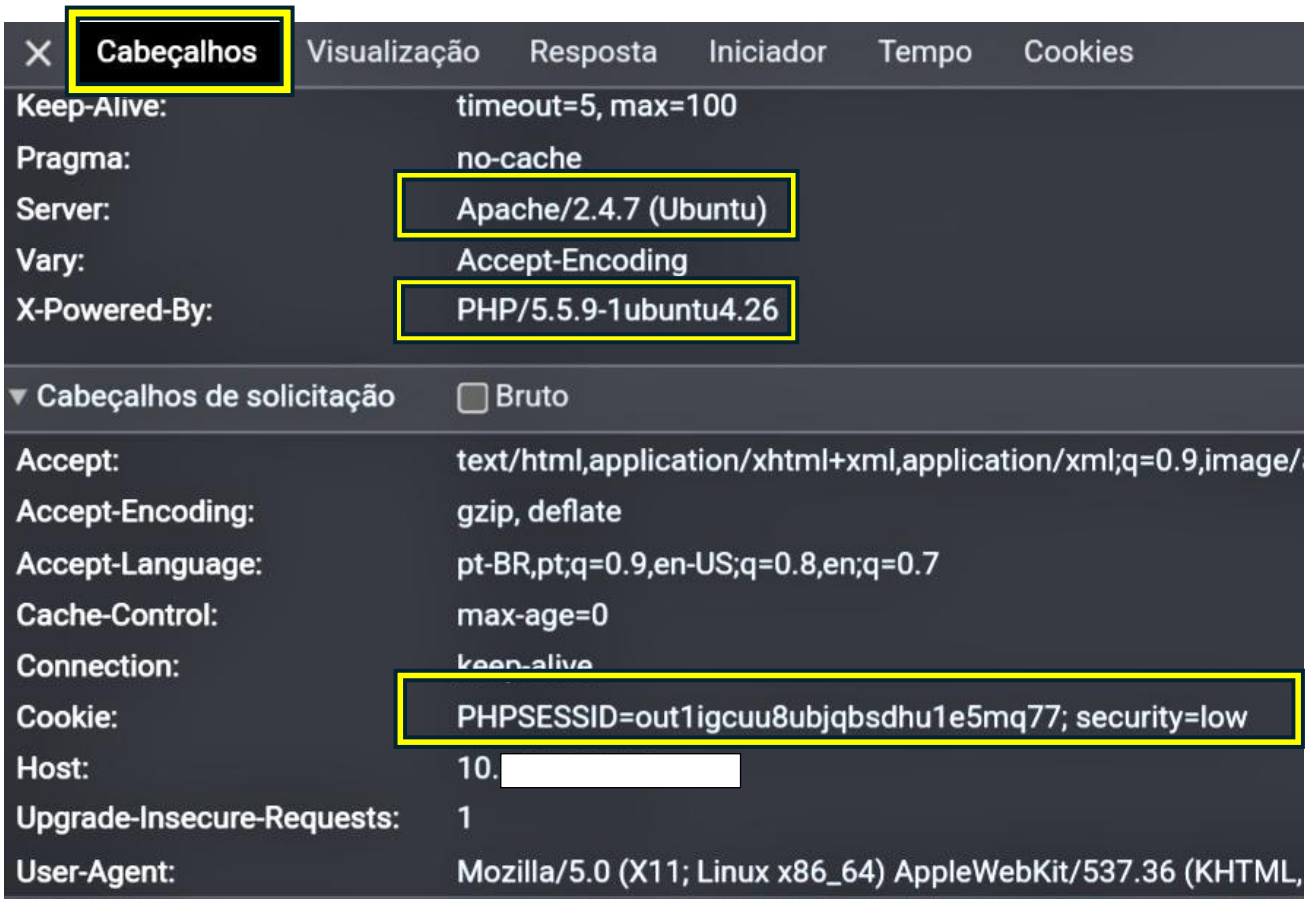

Figura 6 – Captura dos cabeçalhos HTTP via DevTools, evidenciando versão do servidor, tecnologias utilizadas e cookies de sessão.

Cabeçalhos de resposta:

- Keep-Alive: timeout=5, max=100

Indica a manutenção de conexões persistentes entre cliente e servidor, otimizando o desempenho da comunicação.

- Server: Apache/2.4.7 (Ubuntu)

Revela o servidor web e sua versão, permitindo a identificação de tecnologias em uso e possíveis vulnerabilidades conhecidas.

- X-Powered-By: PHP/5.5.9-1ubuntu4.26

Expõe a linguagem e versão utilizada no backend, ampliando a superfície de ataque por meio da divulgação de informações sensíveis.

Cabeçalhos de solicitação:

- Cookie: PHPSESSID=out1ugcuu8ubjbgsdhu1e5mq77; security=low

Indica o uso de sessão ativa e nível de segurança reduzido da aplicação, podendo facilitar ataques como sequestro de sessão.

Medidas

a. Remover ou mascarar cabeçalhos que exponham tecnologias.

b. Configurar cookies com atributos de segurança (HttpOnly e Secure).

c. Fortalecer o gerenciamento de sessões.

6.6 Criação de Wordlist

Descrição

Foi criada uma wordlist personalizada com base em padrões comuns e variações previsíveis de senhas, aumentando a eficácia de ataques de autenticação.

Impacto

O uso de credenciais fracas eleva a probabilidade de comprometimento de contas e acesso indevido ao sistema.

Criticidade

Médio

6.6.1 Evidências:

Figura 7 – Criação e visualização da wordlist personalizada contendo senhas comuns no editor Nano.

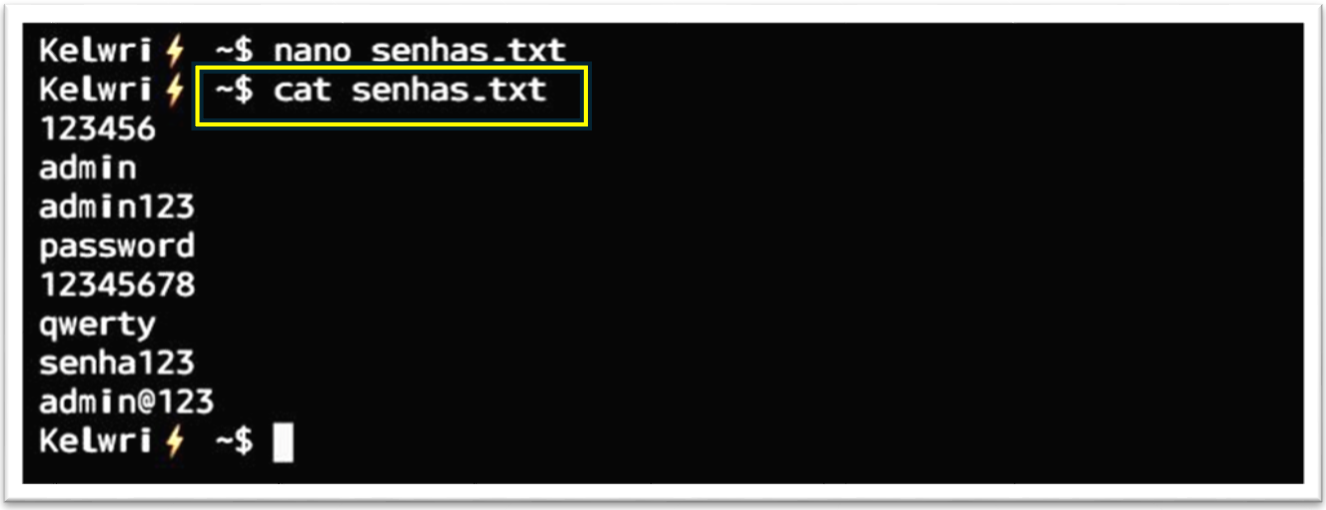

Figura 8 – Exibição do conteúdo da wordlist por meio do comando cat senhas.txt, confirmando as senhas utilizadas no ataque de força bruta.

Medidas

a. Exigir senhas complexas e não previsíveis.

b. Bloquear o uso de senhas comuns ou comprometidas.

c. Implementar autenticação multifator.

6.7 Ataque de Força Bruta com Hydra

Descrição

Foi realizado um ataque de força bruta com a ferramenta Hydra, permitindo múltiplas tentativas automatizadas de autenticação.

Impacto

O ataque resultou na identificação de credenciais válidas, evidenciando a fragilidade do mecanismo de autenticação.

Criticidade

Crítico

6.7.1 Evidências:

Comando utilizado: Hydra -l admin -P senhas.txt 10.XX.XXX.XXX http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=Username and/or password incorrect:H=Cookie: PHPSESSID=out1ugcuu8ubjbgsdhu1e5mq77; security=low"

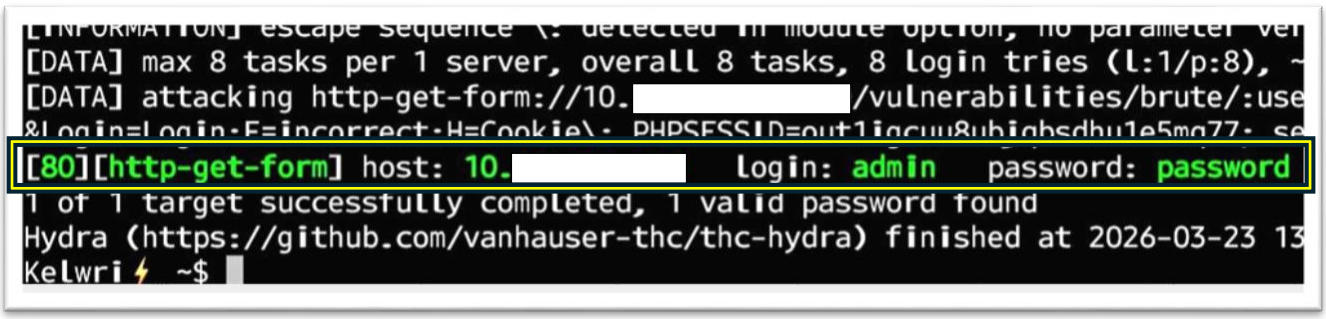

Figura 9 – Execução da ferramenta Hydra identificando credenciais válidas com sucesso.

Resultado:

- **Host:** 10.XX.XXX.XXX

Identificação do alvo testado durante o ataque de força bruta.

- **Usuário:** admin

Conta válida identificada no processo de autenticação.

- **Senha:** password

Credencial comprometida por meio de tentativa automatizada, evidenciando o uso de senha fraca.

- **Status:** sucesso

Foi possível obter acesso válido com 1 combinação de credenciais, confirmando a ausência de mecanismos eficazes de proteção contra força bruta.

Medidas

a. Limitar tentativas de autenticação por período.

b. Bloquear IPs após múltiplas falhas consecutivas.

c. Implementar mecanismos anti-automação.

d. Monitorar eventos de autenticação.

7. Conclusão

O teste de intrusão evidenciou vulnerabilidades relevantes na aplicação, com destaque para falhas no mecanismo de autenticação e ausência de controles de segurança adequados. A exploração por força bruta demonstrou a possibilidade de comprometimento de credenciais e acesso não autorizado. Também foram identificadas fragilidades na configuração de serviços, exposição de informações sensíveis e uso de tecnologias desatualizadas.

Diante desse cenário, recomenda-se a implementação de controles como limitação de tentativas de login, autenticação multifator, configuração segura de cookies e atualização contínua dos serviços. A priorização das vulnerabilidades críticas e de alto risco é essencial para reduzir a superfície de ataque e fortalecer a segurança da aplicação.